

Richtig oder Falsch?

- Eine NTM akzeptiert ein Wort w einer Sprache L , wenn es mindestens einen akzeptierenden Rechenweg gibt.
- Die worst case Laufzeit einer NTM zum Akzeptieren eines Wortes w der Länge n ergibt sich durch den kürzesten akzeptierenden Rechenweg für Worte der Länge n aus der Sprache der NTM.
- Die Klasse P beinhaltet alle Probleme, die in polynomieller Zeit berechnet werden können.
- NP beinhaltet alle Probleme die in nicht-polynomieller Zeit gelöst werden können.

RECAP: Komplexitätstheorie

- Die Komplexitätsklasse P umfasst alle Probleme, die von einer DTM in polynomieller Laufzeit gelöst werden können
- Die Komplexitätsklasse NP umfasst alle Entscheidungsprobleme, die von einer NTM in polynomieller Laufzeit erkannt werden.
- NTM erkennt eine Sprache, wenn es für alle Wörter der Sprache mindestens einen Rechenweg zu einer akzeptierenden Endkonfiguration gibt.
- Zertifikat: potentielle Lösung eines Entscheidungsproblems
- Verifizierer: Algorithmus, der Zertifikat überprüft, um Entscheidungsproblem zu lösen.

$$x \in L \Leftrightarrow \exists y \in \{0,1\}^*, |y| \leq p(|x|): V \text{ akzeptiert } y\#x$$

Komplexitätstheorie

Beispiele für Probleme in NP

- Typische Vertreter für Probleme aus der Klasse NP sind Entscheidungsvarianten von Optimierungsproblemen.
- Beschreibe Optimierungsprobleme durch
 - Eingabe
 - zulässige Lösungen und
 - Zielfunktionen
- Die Ausgabe ist eine zulässige Lösung, die die Zielfunktion optimiert.
 - Bei mehreren optimalen Lösungen, soll genau eine davon ausgegeben werden

Beispiele für Probleme in NP

Problem (**Rucksackproblem**, Knapsack Problem, **KP**)

Beim KP suchen wir eine Teilmenge K von N gegebenen Objekten mit Gewichten $w_1, \dots, w_N$ und Nutzenwerten bzw. Profiten $p_1, \dots, p_N$, so dass die Objekte aus K in einen Rucksack mit Gewichtsschranke b passen und dabei der Nutzen maximiert wird.

- Eingabe: $b \in \mathbb{N}$, $w_1, \dots, w_N \in \{1, \dots, b\}$, $p_1, \dots, p_N \in \mathbb{N}$
- zulässige Lösungen: $K \subseteq \{1, \dots, N\}$ mit $\sum_{i \in K} w_i \leq b$
- Zielfunktion: Maximiere $\sum_{i \in K} p_i$

Beispiele für Probleme in NP

- Zu einem Optimierungsproblem können wir grundsätzlich auch immer eine **Entscheidungsvariante** angeben, in der wir eine Zielvorgabe machen und fragen, ob diese Vorgabe erreicht werden kann.
- Bei der Entscheidungsvariante vom KP ist zur oben beschriebenen Eingabe zusätzlich eine Zahl $p \in \mathbb{N}$ gegeben, und es soll entschieden werden, ob es eine zulässige Teilmenge der Objekte mit Nutzen mindestens p gibt.

Beispiele für Probleme in NP

Problem (**Bin Packing Problem, BPP**)

Beim BPP suchen wir eine Verteilung von N Objekten mit Gewichten $w_1, \dots, w_N$ auf eine möglichst kleine Anzahl von Behältern mit Gewichtskapazität jeweils b . Die Anzahl der Behälter wird mit k bezeichnet.

- Eingabe: $b \in \mathbb{N}$, $w_1, \dots, w_N \in \{1, \dots, b\}$
- zulässige Lösungen: $k \in \mathbb{N}$ und Funktion $f: \{1, \dots, N\} \rightarrow \{1, \dots, k\}$ mit $\forall i \in \{1, \dots, k\}: \sum_{j \in f^{-1}(\{i\})} w_j \leq b$
- Zielfunktion: Minimiere k

Bei der Entscheidungsvariante vom BPP ist $k \in \mathbb{N}$ Teil der Eingabe, und es ist zu entscheiden, ob es eine zulässige Verteilung der Objekte auf höchstens k Behälter gibt.

Beispiele für Probleme in NP

Problem (**Traveling Salesperson Problem, TSP**)

Beim TSP ist ein vollständiger Graph aus N Knoten (Orten) mit Kantengewichten gegeben. Gesucht ist eine **Rundreise** (ein **Hamiltonkreis**, eine **Tour**) mit kleinstmöglichen Kosten.

- Eingabe: $c \in \mathbb{N}^{N \times N}$ symmetrisch
- zulässige Lösungen: Permutationen π von $\{1, \dots, N\}$
- Zielfunktion: Minimiere $c(\pi(N), \pi(1)) + \sum_{i=1}^{N-1} c(\pi(i), \pi(i+1))$

Bei der Entscheidungsvariante vom TSP ist zusätzlich eine Zahl $b \in \mathbb{N}$ gegeben und es ist zu entscheiden, ob es eine Rundreise mit Kosten höchstens b gibt.

Beispiele für Probleme in NP

Satz: Die Entscheidungsvarianten von KP, BPP und TSP sind in NP.

Beweis:

Wir verwenden zulässige Lösungen dieser Probleme als Zertifikat. Dazu müssen wir nachweisen, dass diese Lösungen eine polynomiell in der Eingabelänge beschränkte Kodierungslänge haben und durch einen Algorithmus, dessen Laufzeit ebenfalls polynomiell in der Eingabelänge des Problems ist, verifiziert werden können.

- KP: Die Teilmenge K der ausgewählten Objekte kann mit N Bits kodiert werden. Man überprüft in polynomieller Zeit, ob die Gewichtsschranke b eingehalten und der Nutzenwert p erreicht wird.
- BPP: Die Abbildung $f: \{1, \dots, N\} \rightarrow \{1, \dots, k\}$ kann mit $O(N \cdot \log(k))$ Bits kodiert werden, ist also polynomiell in der Eingabelänge beschränkt. Es kann in polynomieller Zeit überprüft werden, ob die vorgegebene Anzahl Behälter und die Gewichtsschranke je Behälter eingehalten wird.
- TSP: Einer Permutation π kann in $O(N \cdot \log(N))$ Bits kodiert werden. Es kann in polynomieller Zeit überprüft werden, ob die durch π beschriebene Rundreise die vorgegebene Kostenschranke b einhält.

QED

Varianten von Problemen

- Aus der Entscheidungsvariante kann man üblicherweise
 - den bestmöglichen Wert über Binärsuche effizient ausrechnen und
 - das Optimierungsproblem lösen, indem man Objekte/Kanten/Knoten/etc. löscht und schaut, ob diese den bestmöglichen Wert verschlechtern.

Satz: Wenn die Entscheidungsvariante von KP in polynomieller Zeit lösbar ist, dann auch die Optimierungsvariante.

Beweis:

Neben der Entscheidungs- und Optimierungsvariante betrachten wir die folgende Zwischenvariante: Gesucht ist der Zielfunktionswert einer optimalen Lösung, ohne dass die optimale Lösung selber ausgegeben werden muss.

Varianten von Problemen

Beweis (cont.)

Aus einem Algorithmus A für die Entscheidungsvariante, konstruieren wir zunächst einen Algorithmus B für die Zwischenvariante. Algorithmus B berechnet

- den maximal möglichen Profit p , indem er eine Binärsuche über dem Wertebereich $\{0, \dots, P\}$ ausführt, wobei $P = \sum p_i$ eine triviale obere Schranke für p ist.
- In der Binärsuche werden die Entscheidungen über die "Hälfte" des Wertebereichs auf der die Suche fortgesetzt wird mit Hilfe von Algorithmus A getroffen.

Die Binärsuche hat einem Wertebereich der Größe $P + 1$.

Falls $P + 1$ eine Zweierpotenz ist, so benötigt die Suche genau $\log_2(P + 1)$ Iterationen, da sich der Wertebereich von Iteration zu Iteration halbiert.

Ist der Wertebereich keine Zweierpotenz, so gilt eine obere Schranke von $\lceil \log_2(P + 1) \rceil$ für die Anzahl Aufrufe von Algorithmus A .

Varianten von Problemen

Beweis (cont.) Setze diese Laufzeitschranke in Beziehung zur Eingabelänge.

Wir nehmen an, alle Eingabezahlen werden auf kanonische Art binär kodiert.

Die Kodierungslänge von $a \in \mathbb{N}$ bezeichnen wir mit $\kappa(a)$.

Es gilt $\kappa(a) = \lceil \log(a + 1) \rceil$.

Die Binärdarstellung der Summe zweier Zahlen ist kürzer als die Summe der Längen ihrer Binärdarstellungen, d.h. für $a, b \in \mathbb{N}$ gilt $\kappa(a + b) \leq \kappa(a) + \kappa(b)$. Die Eingabelänge ist also mindestens

$$n \geq \sum_{i=1}^N \kappa(p_i) \geq \kappa(\sum_{i=1}^N p_i) = \kappa(P) = \lceil \log_2(P + 1) \rceil.$$

Der letzte Term entspricht genau der oberen Schranke für die Anzahl Iterationen der Binärsuche. Die Anzahl der Aufrufe von Algorithmus A ist somit linear in der Eingabelänge beschränkt.

Falls also die Laufzeit von A polynomiell beschränkt ist, so ist auch B ein Polynomialzeitalgorithmus.

Varianten von Problemen

Beweis (cont.):

Als nächstes zeigen wir, wie aus dem Algorithmus B für die Zwischenvariante, ein Algorithmus C für die Optimierungsvariante konstruiert werden kann.

Der Algorithmus betrachtet die Objekte nacheinander. Für jedes Objekt $i \in \{1, \dots, N\}$ testet er mit Hilfe von Algorithmus B , ob es auch eine optimale Lösung ohne Objekt i gibt. Falls Objekt i nicht für die optimale Lösung benötigt wird, so wird es gestrichen.

Auf diese Art und Weise bleibt am Ende eine Menge von Objekten übrig, die zulässig ist, also die Gewichtsschranke einhält, und den optimalen Zielfunktionswert erreicht.

Die Laufzeit von Algorithmus C ist im Wesentlichen durch die $N + 1$ Unterprogrammaufrufe für Algorithmus B bestimmt.

Falls also die Laufzeit von B polynomiell beschränkt ist, so ist auch C ein Polynomialzeitalgorithmus.

P versus NP

Das wohl bekannteste offene Problem in der Informatik ist die Frage

$$P = NP ?$$

Gehört zur Liste der Millenium-Probleme (Liste der ungelösten Probleme der Mathematik) des Clay Mathematics Institute, Cambridge (Massachusetts)

Preis für die Lösung: eine Millionen Dollar und ewiger Ruhm.

P versus NP

Frage $P=NP$? macht in dieser Form wenig Sinn, denn wir haben P als Menge von **Problemen** (z.B. Sortieren) und NP als Menge von **Entscheidungsproblemen** (Sprachen) definiert.

Wenn E die Menge aller Entscheidungsprobleme ist, so müsste die Frage eigentlich lauten $P \cap E = NP$?

Die Bezeichnung P wird doppeldeutig verwendet, mal für alle Probleme, die einen Polynomialzeitalgorithmus haben, und mal für Entscheidungsprobleme, die in polynomieller Zeit entschieden werden können.

Vereinbarung: wenn wir P mit NP (oder ähnlichen Klassen) vergleichen, so schränken wir P auf die Klasse der Entscheidungsprobleme ein.

P versus NP

Da eine deterministische TM auch als eine nichtdeterministische TM aufgefasst werden kann, die vom Nichtdeterminismus keinen Gebrauch macht, gilt offensichtlich

$$P \subseteq NP$$

P versus NP

- Die Klasse NP scheint wesentlich mächtiger als die Klasse P zu sein, da die NTM die magische Fähigkeit hat, den richtigen Rechenweg aus einer Vielzahl von Rechenwegen zu raten.
- Gesehen: NP sind Entscheidungsprobleme mit einem Polynomialzeitalgorithmus, der ein Zertifikat umsonst bekommt und nur die "JA-Antworten" verifizieren muss.
- Simuliere Nichtdeterminismus durch Ausprobieren aller Zertifikate.
 - Die Probleme in NP sind rekursiv.
 - Liefert eine exponentielle Laufzeitschranke.

Satz: Für jedes Entscheidungsproblem $L \in \text{NP}$ gibt es einen Algorithmus A , der L entscheidet, und dessen worst case Laufzeit durch $2^{q(n)}$ nach oben beschränkt ist, wobei q ein geeignetes Polynom ist.

P versus NP

Beweis:

Der Verifizierer V für L (Verifiziert $y\#x$ für Zertifikat y) und das Polynom p (obere Schranke Länge des Zertifikats) seien definiert wie oben.

Um die Eingabe $x \in \{0,1\}^n$ zu entscheiden, generiert Algorithmus A nacheinander alle möglichen Zertifikate $y \in \{0,1\}^{p(n)}$ und startet den Verifizierer V mit der Eingabe $y\#x$.

Falls V eines der generierten Zertifikate akzeptiert, so akzeptiert auch A . Falls V keines der Zertifikate akzeptiert, so verwirft A .

Die Korrektheit von A folgt direkt aus obigem Satz über Zertifikate.

QED

P versus NP

Analysiere die Laufzeit von A :

- Sei p' eine polynomielle Laufzeitschranke für den Verifizierer V .
- Definiere das Polynom $q(n) = p(n) + p'(p(n) + 1 + n)$.
- Die Laufzeit von Algorithmus A ist nach oben beschränkt durch:

$$2^{p(n)} \cdot p'(p(n) + 1 + n) \leq 2^{p(n)} \cdot 2^{p'(p(n)+1+n)} = 2^{q(n)}$$

QED

P versus NP

Wir können die Frage „ $P = NP$?“ auch folgendermaßen interpretieren:

- Ist das Herleiten eines Beweises (Zertifikates) wesentlich schwieriger als das Nachvollziehen eines gegebenen Beweises?
- Möglicherweise hat das Herleiten exponentielle Komplexität während das Nachvollziehen in polynomieller Zeit möglich ist.
- Mehrheitsmeinung: Es ist schwieriger einen Beweis herzuleiten als ihn nachzuvollziehen.

Also Arbeitshypothese: $P \neq NP$.

P versus NP

- Die Klasse NP enthält viele Probleme, für die es trotz intensivster Bemühungen bis heute nicht gelungen ist, einen Polynomialzeitalgorithmus zu entwickeln.
- Insbesondere in der Optimierung treten immer wieder scheinbar harmlose Probleme mit Anwendungen zum Beispiel aus den Bereichen Produktion, Logistik und Ökonomie auf, die auch tatsächlich einfach zu lösen wären, hätten wir einen Rechner, der mit so etwas wie "Intuition" ausgestattet wäre, um den richtigen Rechenweg aus einer Vielzahl von Rechenwegen zu "erraten".
- Leider gibt es aber derartige Rechner nur in theoretischen Modellen, wie etwa dem Modell der nichtdeterministischen Turingmaschine.
- Derartig verschrobene theoretische Modelle können uns bei der praktischen Arbeit aber wohl kaum behilflich sein, oder vielleicht doch?

NP Vollständigkeit

Beispiel: Traveling Salesman Problem (TSP)

- Bisher (trotz großer Bemühungen) kein Polynomialzeitalgorithmus bekannt. Aber wir haben effiziente Algorithmen für scheinbar sehr ähnliche Problemstellung wie Kürzeste-Wege (Dijkstra) oder dem Minimalen-Spannbaum (Prim, Kruskal, ...).
- Frage: Können wir begründen, warum wir keinen Polynomialzeitalgorithmus finden? (Hilft auch bei Anfragen von Kunden oder Management.)
- Die **NP-Vollständigkeitstheorie** liefert eine derartige Begründung. Sie erlaubt es uns, Probleme bezüglich ihrer Schwierigkeit miteinander zu vergleichen.
- Wir werden zeigen, dass beispielsweise TSP zu den schwierigsten Problemen in NP gehört. Unter der Hypothese $P \neq NP$ hat TSP deshalb beweisbar keinen Polynomialzeitalgorithmus.
- Wir werden sehen, dass TSP keine Ausnahme ist, sondern diese Eigenschaft mit zahlreichen anderen Problemen aus NP teilt, den sogenannten "NP-vollständigen" Problemen.

NP Vollständigkeit

- L_1 und L_2 seien zwei Sprachen über Σ_1 bzw. Σ_2 . Dann ist L_1 **polynomiell reduzierbar** auf L_2 , wenn es eine Reduktion von L_1 nach L_2 gibt, die in polynomieller Zeit berechenbar ist.
- Wir schreiben $L_1 \leq_p L_2$.
- Erinnerung: Eine Reduktion $L_1 \leq L_2$ ist definiert durch eine berechenbare Funktion $f: \Sigma_1^* \rightarrow \Sigma_2^*$, so dass für all $x \in \Sigma_1^*$ gilt: $x \in L_1 \Leftrightarrow f(x) \in L_2$.
- Bei der **Polynomialzeitreduktion** $L_1 \leq_p L_2$ verlangen wir also lediglich zusätzlich, dass f in polynomieller Zeit berechnet werden kann.
- Intuitiv: $L_1 \leq_p L_2$ bedeutet „ L_1 ist nicht schwieriger als L_2 “, modulo polynomieller Eingabetransformation.

NP Vollständigkeit

Lemma $L_1 \leq_p L_2, L_2 \in P \Rightarrow L_1 \in P$.

Beweis:

Sei A ein Polynomialzeitalgorithmus für L_2 und f eine Funktion mit $x \in L_1 \Leftrightarrow f(x) \in L_2$.

Entwerfe Algorithmus B für L_1 :

- B berechnet aus der Eingabe x für L_1 die Eingabe $f(x)$ für L_2 und
- startet A auf $f(x)$ und übernimmt das Akzeptanzverhalten.

Die Korrektheit von B folgt direkt aus der Eigenschaft $x \in L_1 \Leftrightarrow f(x) \in L_2$.

NP Vollständigkeit

Polynomielle Laufzeit von B :

- Sei p ein Polynom, dass die Laufzeit für die Berechnung von f beschränkt.
- Es gilt $|f(x)| \leq p(|x|)$
- Die Laufzeit von A ist durch ein Polynom q in der Länge von $f(x)$ beschränkt. Wir können q als monoton wachsend annehmen.
- Wir erhalten die Laufzeitschranke für Algorithmus B :

$$p(|x|) + q(|f(x)|) \leq p(|x|) + q(p(|x|))$$

NP-harte Probleme

Ein Problem L heißt **NP-hart**, falls gilt $\forall L' \in \text{NP}: L' \leq_p L$.
 (Im Englischen NP-hard, man findet auch die Übersetzung NP-schwer.)

Satz: Sei $L \in P$ und L NP-hart $\Rightarrow P = \text{NP}$.

Beweis:

Sei L' ein beliebiges Problem aus NP. Da L NP-hart ist, gilt $L' \leq_p L$.

Aus $L \in P$ folgt wegen des Lemmas, dass $L' \in P$.

Also ist jedes Problem aus NP auch in P enthalten, so dass gilt $P = \text{NP}$.

QED

Unter der Hypothese $P \neq \text{NP}$ gibt es für NP-harte Probleme deshalb keinen Polynomialzeitalgorithmus.

NP Vollständigkeit

- Ein Problem L heißt **NP-vollständig** (NP-complete), falls gilt $L \in \text{NP}$ und L ist NP-hart.
 - Intuitiv: dies sind die schwierigsten unter den Problemen in NP.
- NPC bezeichnet die Klasse der **NP-vollständigen Probleme**.
 - Alle Probleme in NPC können als gleich schwierig modulo polynomieller Eingabetransformationen angesehen werden.

NP-hart vs. NP-vollständig

- Worin liegt der Unterschied zwischen NP-hart und NP-vollständig?
- NP-hart
Ein Problem L heißt **NP-hart**, falls gilt $\forall L' \in \text{NP}: L' \leq_p L$.
- NP-vollständig
Ein Problem L heißt **NP-vollständig** (NP-complete), falls gilt $L \in \text{NP}$ und L ist NP-hart.

Komplexitätsklassen

- Die Komplexitätsklassen P und NP können in die Chomsky-Hierarchie folgendermaßen eingeordnet werden:

